

SPRINT 3 – BRIEF 4

AffiliateLink: Plateforme d’affiliation

Tests des modules profile et offer

Contexte du brief

Les modules profile et offer contiennent maintenant des règles métier qui touchent directement à l'argent des advertisers et des publishers : modèles de rémunération et montants, transitions de statut, une seule campagne active par offre, mise en pause des campagnes à la suspension d'un compte. Jusqu'ici, ces règles ont été vérifiées à la main, avec Postman : chaque modification risque d'en casser une sans que personne ne s'en aperçoive.

Avant d'ajouter la sécurité au sprint suivant, l'équipe veut un **filet de sécurité** : des **tests automatisés** qui vérifient les règles à chaque build, à tous les niveaux de l'application, une **mesure de la couverture** pour voir ce qui n'est pas testé, et une première **analyse de la qualité** du code.

Spécifications fonctionnelles

Les tests couvrent au minimum les comportements suivants.

Module profile

- Un profil créé a le statut PENDING.
- Un email déjà utilisé est refusé (409) et aucun profil n'est créé.
- La suspension d'un compte publie l'événement `ProfileSuspended` ; son activation n'en publie aucun.
- Un profil inexistant renvoie 404.

Module offer

- Toutes les transitions de statut d'une campagne, autorisées et interdites.
- Une campagne dont la date de fin est dépassée ne peut pas être activée.
- Les règles de rémunération : montant > 0 (CPC, CPL, CPA_FIXED), pourcentage entre 0 et 100 (CPA_PERCENT), date de fin après la date de début.
- Activer une campagne met en pause la campagne active de la même offre.
- Une offre est refusée si l'advertiser est inexistant ou non actif.
- Les tags sont enregistrés en minuscules, sans doublon, et partagés entre les offres.
- La recherche d'offres : chaque critère seul, une combinaison, aucun critère.
- La pagination et le tri des offres.

Entre les modules

- La suspension d'un advertiser met ses campagnes actives en PAUSED.

Spécifications techniques

Niveaux de tests

Niveau	Cible	Outils
Unitaire	entités et services, dépendances mockées	JUnit 5, Mockito, AssertJ
Repository	requêtes JPQL, Specifications, pagination	`@DataJpaTest`, Testcontainers
Contrôleur	un test par contrôleur : 201, 400, 404, 409	`@WebMvcTest`, `@MockitoBean`
Module	`verify()`, écoute de `ProfileSuspended`	`@ApplicationModuleTest`
Intégration complète	parcours de bout en bout, sans mock	`@SpringBootTest`, MockMvc, Testcontainers

Règles par niveau

- Services testés sans Spring ni base de données, toutes leurs dépendances mockées avec Mockito.
- Transitions de statut testées avec `@ParameterizedTest`.
- Codes HTTP 201, 400, 404 et 409 vérifiés, avec le contenu du `ProblemDetail` pour les erreurs.

Tests d'intégration complets

- Inscription d'un advertiser, puis consultation de son profil.
- Double inscription avec le même email.
- Parcours d'une campagne : création d'une offre et de deux campagnes, activations successives.
- Recherche paginée des offres.
- Suspension d'un advertiser et mise en pause de ses campagnes.

Couverture et qualité

- JaCoCo : rapport généré par `mvn verify`, seuil minimal de **60 % des lignes** ; classe principale, configuration, DTO et code Lombok exclus.
- SonarQube en local (Docker) : **aucun bug ni vulnérabilité**.

Exigences

- Nommage : `*Test` pour les tests unitaires, `*IT` pour les tests qui démarrent Spring ou une base de données.
- Structure Given / When / Then ; nom de test qui décrit le comportement attendu.
- Tests indépendants : aucune dépendance aux données ou à l'ordre d'exécution des autres tests.
- `mvn clean verify` passe sur un poste disposant uniquement de Java et Docker.